

Pharma Billing — Full Backend + Frontend Blueprint

This document bundles: folder structure, Mongoose schemas, invoice controller (transactional), routes & middleware, ERD (ASCII), Postman snippets, and a React frontend skeleton. Use it as a single-source implementation plan — copy files into your project and iterate.

1. Project folder structure

```
pharma-billing/
├─ backend/
│  └─ src/
│     ├── config/
│     │   └─ index.js
│     ├── controllers/
│     │   ├── auth.controller.js
│     │   ├── user.controller.js
│     │   ├── item.controller.js
│     │   ├── batch.controller.js
│     │   └─ invoice.controller.js
│     ├── models/
│     │   ├── user.model.js
│     │   ├── customer.model.js
│     │   ├── vendor.model.js
│     │   ├── item.model.js
│     │   ├── batch.model.js
│     │   ├── invoice.model.js
│     │   ├── ledger.model.js
│     │   └─ audit.model.js
│     ├── routes/
│     │   ├── auth.routes.js
│     │   ├── user.routes.js
│     │   ├── item.routes.js
│     │   └─ invoice.routes.js
│     ├── services/
│     │   ├── stock.service.js
│     │   ├── ledger.service.js
│     │   └─ audit.service.js
│     ├── middlewares/
│     │   ├── auth.middleware.js
│     │   └─ role.middleware.js
│     └─ app.js
└─ package.json
├─ frontend/
│  └─ src/
```

```
|   |   | api/
|   |   |   | api.js
|   |   | components/
|   |   |   | Auth/
|   |   |   | Dashboard/
|   |   |   | Inventory/
|   |   |   |   | Billing/
|   |   | pages/
|   |   |   | Login.jsx
|   |   |   | Dashboard.jsx
|   |   |   | Inventory.jsx
|   |   |   |   | Billing.jsx
|   |   | context/
|   |   |   | authContext.jsx
|   |   | App.jsx
|   | package.json
```

2. package.json snippets

backend/package.json minimal:

```
{
  "name": "pharma-backend",
  "version": "1.0.0",
  "main": "src/app.js",
  "scripts": {
    "start": "node src/app.js",
    "dev": "nodemon src/app.js"
  },
  "dependencies": {
    "bcrypt": "^5.1.0",
    "express": "^4.18.2",
    "jsonwebtoken": "^9.0.0",
    "mongoose": "^7.0.0",
    "cors": "^2.8.5",
    "helmet": "^7.0.0",
    "winston": "^3.8.0"
  }
}
```

3. Mongoose Schemas (ready-to-use)

user.model.js

```
const mongoose = require('mongoose')
const Schema = mongoose.Schema

const UserSchema = new Schema({
  username: { type: String, required: true, unique: true },
  name: String,
  email: { type: String, unique: true, sparse: true },
  passwordHash: { type: String, required: true },
  role: { type: String, enum: ['admin', 'staff'], required: true },
  active: { type: Boolean, default: true },
  createdBy: { type: Schema.Types.ObjectId, ref: 'User' }
}, { timestamps: true })

module.exports = mongoose.model('User', UserSchema)
```

customer.model.js

```
const mongoose = require('mongoose')
const Schema = mongoose.Schema

const CustomerSchema = new Schema({
  name: { type: String, required: true },
  phone: String,
  address: String,
  gstNumber: String,
  balance: { type: Number, default: 0 }
}, { timestamps: true })

module.exports = mongoose.model('Customer', CustomerSchema)
```

vendor.model.js

```
const mongoose = require('mongoose')
const Schema = mongoose.Schema

const VendorSchema = new Schema({
  name: { type: String, required: true },
  phone: String,
  address: String,
  gstNumber: String
}, { timestamps: true })

module.exports = mongoose.model('Vendor', VendorSchema)
```

item.model.js

```
const mongoose = require('mongoose')
const Schema = mongoose.Schema

const ItemSchema = new Schema({
  sku: { type: String, unique: true, index: true },
  name: { type: String, required: true },
  brand: String,
  hsnCode: String,
  packSize: String,
  mrp: Number,
  purchaseRate: Number,
  saleRate: Number,
  gstPercent: Number,
  unit: String,
  reorderLevel: { type: Number, default: 0 },
  stock: { type: Number, default: 0 }
}, { timestamps: true })

module.exports = mongoose.model('Item', ItemSchema)
```

batch.model.js (InventoryBatch)

```
const mongoose = require('mongoose')
const Schema = mongoose.Schema

const BatchSchema = new Schema({
  itemId: { type: Schema.Types.ObjectId, ref: 'Item', required: true, index: true },
  batchNumber: String,
  quantityAvailable: { type: Number, default: 0 },
  quantityTotal: { type: Number, default: 0 },
  purchaseRate: Number,
  mrp: Number,
  manufactureDate: Date,
  expiryDate: Date,
  vendorId: { type: Schema.Types.ObjectId, ref: 'Vendor' },
  expired: { type: Boolean, default: false }
}, { timestamps: true })

module.exports = mongoose.model('InventoryBatch', BatchSchema)
```

invoice.model.js

```
const mongoose = require('mongoose')
const Schema = mongoose.Schema
```

```

const InvoiceItemSchema = new Schema({
  itemId: { type: Schema.Types.ObjectId, ref: 'Item' },
  allocations: [
    { batchId: { type: Schema.Types.ObjectId, ref: 'InventoryBatch' }, qty:
Number }
  ],
  qty: Number,
  rate: Number,
  discount: { type: Number, default: 0 },
  gstPercent: Number,
  lineTotal: Number
})

const InvoiceSchema = new Schema({
  invoiceNumber: { type: String, index: true, unique: true },
  customerId: { type: Schema.Types.ObjectId, ref: 'Customer' },
  createdBy: { type: Schema.Types.ObjectId, ref: 'User' },
  date: { type: Date, default: Date.now },
  items: [InvoiceItemSchema],
  subTotal: Number,
  totalGST: Number,
  totalAmount: Number,
  paymentType: { type: String, enum: ['cash','card','credit','upi'] },
  paymentStatus: { type: String, enum: ['paid','pending','partial'],
default: 'paid' },
  stockImpactProcessed: { type: Boolean, default: false }
}, { timestamps: true })

module.exports = mongoose.model('Invoice', InvoiceSchema)

```

ledger.model.js

```

const mongoose = require('mongoose')
const Schema = mongoose.Schema

const LedgerEntrySchema = new Schema({
  type: { type: String },
  refId: Schema.Types.ObjectId,
  date: { type: Date, default: Date.now },
  entries: [ { account: String, debit: Number, credit: Number } ],
  createdBy: { type: Schema.Types.ObjectId, ref: 'User' }
}, { timestamps: true })

module.exports = mongoose.model('LedgerEntry', LedgerEntrySchema)

```

audit.model.js

```

const mongoose = require('mongoose')
const Schema = mongoose.Schema

```

```
const AuditSchema = new Schema({
  userId: { type: Schema.Types.ObjectId, ref: 'User' },
  actionType: String,
  collectionName: String,
  documentId: Schema.Types.ObjectId,
  before: Schema.Types.Mixed,
  after: Schema.Types.Mixed,
  ip: String
}, { timestamps: true })

module.exports = mongoose.model('AuditLog', AuditSchema)
```

4. Core services (short)

stock.service.js — allocation logic (FEFO)

```
const InventoryBatch = require('../models/batch.model')

async function allocateStock(session, itemId, qtyNeeded) {
  const allocations = []
  const batches = await InventoryBatch.find({
    itemId,
    quantityAvailable: { $gt: 0 },
    expiryDate: { $gt: new Date() }
  }).sort({ expiryDate: 1 }).session(session)

  for (const b of batches) {
    if (qtyNeeded <= 0) break
    const take = Math.min(b.quantityAvailable, qtyNeeded)
    b.quantityAvailable -= take
    await b.save({ session })
    allocations.push({ batchId: b._id, qty: take })
    qtyNeeded -= take
  }

  if (qtyNeeded > 0) {
    throw new Error('INSUFFICIENT_STOCK')
  }

  return allocations
}

module.exports = { allocateStock }
```

5. Invoice controller (full, transactional)

invoice.controller.js

```
const mongoose = require('mongoose')
const Invoice = require('../models/invoice.model')
const InventoryBatch = require('../models/batch.model')
const Item = require('../models/item.model')
const LedgerService = require('../services/ledger.service')
const StockService = require('../services/stock.service')

async function createInvoice(req, res) {
  const session = await mongoose.startSession()
  session.startTransaction()
  try {
    const { customerId, items, paymentType } = req.body
    if (!items || !items.length) return res.status(400).send({ message: 'No items' })

    const invoiceItems = []
    let subTotal = 0
    let totalGST = 0

    // 1. allocate stock for each item
    for (const line of items) {
      const item = await Item.findById(line.itemId).session(session)
      if (!item) throw new Error('ITEM_NOT_FOUND')

      const allocations = await StockService.allocateStock(session,
line.itemId, line.qty)

      const rate = line.rate ?? item.saleRate ?? item.mrp
      const lineAmount = rate * line.qty - (line.discount || 0)
      const gst = (lineAmount * (item.gstPercent || 0)) / 100

      invoiceItems.push({
        itemId: item._id,
        allocations,
        qty: line.qty,
        rate,
        discount: line.discount || 0,
        gstPercent: item.gstPercent || 0,
        lineTotal: lineAmount
      })

      subTotal += lineAmount
      totalGST += gst
    }

    const totalAmount = subTotal + totalGST
```

```

// 2. create invoice
const invoiceNumber = `INV-${Date.now()}`
const invoice = new Invoice({
  invoiceNumber,
  customerId,
  createdBy: req.user._id,
  items: invoiceItems,
  subTotal,
  totalGST,
  totalAmount,
  paymentType,
  paymentStatus: paymentType === 'credit' ? 'pending' : 'paid',
  stockImpactProcessed: true
})

await invoice.save({ session })

// 3. create ledger entry (simplified)
await LedgerService.createSaleEntry(session, invoice, req.user._id)

await session.commitTransaction()
session.endSession()

return res.status(201).send(invoice)
} catch (err) {
  await session.abortTransaction()
  session.endSession()
  console.error(err)
  if (err.message === 'INSUFFICIENT_STOCK') return res.status(400).send({
    message: 'Insufficient stock' })
  return res.status(500).send({ message: err.message })
}
}

module.exports = { createInvoice }

```

6. Ledger service (simplified)

ledger.service.js

```

const LedgerEntry = require('../models/ledger.model')

async function createSaleEntry(session, invoice, createdBy) {
  const entries = []
  // Sales credit
  entries.push({ account: 'Sales', credit: invoice.subTotal, debit: 0 })

```



```

// GST credit
if (invoice.totalGST && invoice.totalGST > 0) {
  entries.push({ account: 'GST', credit: invoice.totalGST, debit: 0 })
}

// Cash / Customer debit
const debitAccount = invoice.paymentType === 'credit' ? 'Customer' : 'Cash'
entries.push({ account: debitAccount, debit: invoice.totalAmount, credit:
0 })

const ledger = new LedgerEntry({ type: 'sale', refId: invoice._id,
entries, createdBy })
await ledger.save({ session })
}

module.exports = { createSaleEntry }

```

7. Middleware (auth & role)

auth.middleware.js

```

const jwt = require('jsonwebtoken')
const User = require('../models/user.model')

module.exports = async function (req, res, next) {
  try {
    const auth = req.headers.authorization
    if (!auth) return res.status(401).send({ message: 'No token' })
    const token = auth.split(' ')[1]
    const payload = jwt.verify(token, process.env.JWT_SECRET)
    const user = await User.findById(payload.userId)
    if (!user) return res.status(401).send({ message: 'Invalid token' })
    req.user = user
    next()
  } catch (err) {
    return res.status(401).send({ message: 'Unauthorized' })
  }
}

```

role.middleware.js

```

module.exports = function(allowedRoles = []){
  return (req,res,next) =>{
    if(!allowedRoles.includes(req.user.role)) return res.status(403).send({
message: 'Forbidden' })
    next()
  }
}

```

```
}  
}
```

8. Routes (examples)

invoice.routes.js

```
const express = require('express')  
const router = express.Router()  
const InvoiceController = require('../controllers/invoice.controller')  
const auth = require('../middlewares/auth.middleware')  
const role = require('../middlewares/role.middleware')  
  
router.post('/', auth, role(['admin', 'staff']),  
  InvoiceController.createInvoice)  
router.get('/:id', auth, role(['admin', 'staff']),  
  InvoiceController.getInvoice)  
  
module.exports = router
```

9. ERD (ASCII) — simplified

```
[User] 1---* [Invoice]  
[Customer] 1---* [Invoice]  
[Item] 1---* [InventoryBatch]  
[InventoryBatch] *---* [InvoiceItem.allocations] (via batchId)  
[Invoice] 1---* [InvoiceItem]  
[Invoice] 1---1 [LedgerEntry]
```

10. Postman examples (collection snippet)

```
{  
  "info": { "name": "Pharma API" },  
  "item": [  
    { "name": "Login", "request": { "method": "POST", "header": [], "body":  
    { "raw": "{ \"username\": \"admin\", \"password\": \"pass\"}" } , "url": {  
    "raw": "http://localhost:3000/api/auth/login" } } },  
    { "name": "Create Invoice", "request": { "method": "POST", "header": [ {  
    "key": "Authorization", "value": "Bearer {{token}}" } ], "body": { "raw":  
    "{ \"customerId\": \"...\", \"items\": [{ \"itemId\": \"...\", \"qty\": 2 } ]" },  
    "url": { "raw": "http://localhost:3000/api/invoices" } } }  
  ]  
}
```

```
]
}
```

11. React frontend skeleton

- `pages/Login.jsx` — login form posts to `/api/auth/login`, stores token
- `context/authContext.jsx` — provides user and token
- `pages/Billing.jsx` — search item by SKU, add items to cart, show available batches (FEFO), create invoice via API
- `components/Inventory/BatchList.jsx` — show batches with expiry, quantity, and actions
- `components/Dashboard/Reports.jsx` — fetch `/api/reports/*`

Small example: `src/api/api.js`

```
import axios from 'axios'
const api = axios.create({ baseURL: process.env.REACT_APP_API })
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
export default api
```

12. Extras & Notes

- Use MongoDB transactions (replica set required) for invoice create & stock updates.
- FEFO allocation implemented in `stock.service.js` — change to FIFO if desired.
- Always validate user input (Joi/Zod) in controllers.
- Add tests for concurrency: simulate 2 concurrent invoices for same low-stock item.
- Add scheduled job (node-cron) to run expiry checks and create adjustment ledger entries.

13. Next steps I can do for you (pick any):

- Generate each backend file as separate code files (I can produce them one-by-one)
- Create a Postman collection JSON file you can import
- Produce a minimal React app (single-file) demonstrating Billing flow
- Provide Dockerfile and docker-compose for local dev

End of document.